MEDIACREST
TRAINING COLLEGE

Mediacrest Training College

Nairobi, Kenya

Advanced Data Analytics & Machine Learning

8-Week Advanced Training Program

Comprehensive Session Notes

Week 6 · Day 1

Advanced Data Balancing Techniques (SMOTE) & Ensemble Model Evaluation

Course Trainer: Victor Wandera Lumumba, MSc.
Role: Statistician, Data Analyst & ML Expert

Session Date: September 2026
Mode: Hybrid – Online / Physical (EAT)

"In clinical machine learning, accuracy is a dangerous illusion. True model robustness is forged in the fires of class imbalance, where Synthetic Minority Over-sampling (SMOTE) and leak-proof pipelines separate amateur scripts from production-grade diagnostic tools."

About This Document

These comprehensive notes accompany **Week 6, Day 1** of the **Advanced Data Analytics & Machine Learning** program at Mediacrest Training College. Building upon the foundational ensemble classifiers explored in previous sessions, this module dives deep into the critical challenge of **Class Imbalance** and the rigorous implementation of **Synthetic Minority Over-sampling Technique (SMOTE)**.

We will rigorously explore the mathematical foundations of SMOTE, the catastrophic risks of data leakage during cross-validation, and the "double-balancing" pitfall. Furthermore, we will bridge these advanced preprocessing techniques with rigorous clinical evaluation metrics (ROC-AUC, PR-AUC, F1-Score, False Negative Rate) using the 'Heart.csv' dataset, ensuring models are not only statistically sound but also safe and interpretable for healthcare deployment.

Course at a Glance

Duration	8 Weeks (20+ intensive sessions)
Core Tools	Python (pandas, scikit-learn, imbalanced-learn, XGBoost)
Focus	SMOTE mathematics, leak-proof pipelines, clinical metric trade-offs
Capstone	Production-ready ML system with diagnostics, explainability & policy recommendations
Context	Healthcare, Clinical Risk Prediction, and Epidemiological Modeling

How to Use These Notes

- **Master the Math:** The **Mathematical Foundation** boxes contain rigorous derivations essential for understanding *why* SMOTE behaves the way it does and how it alters decision boundaries.
- **Follow along in code:** Every listing is runnable. Open the companion Jupyter Notebook and execute each cell as you read to observe the dynamic changes in class distribution.
- **Contextualize:** **Kenyan Clinical Context** boxes connect abstract math to local healthcare realities (e.g., MOH policy, resource constraints, false negative costs).

Contents

About This Document 1

1 The Clinical Reality of Class Imbalance 3
- 1.1 The Accuracy Paradox . . . 3
- 1.2 Why Standard Algorithms Fail on Imbalanced Data . . . 3

2 Mathematical Foundations of Imbalanced Learning 3
- 2.1 Prior Probability Shift . . . 4
- 2.2 The Goal of Balancing . . . 4

3 Data-Level Resampling Techniques 4
- 3.1 Random Undersampling . . . 4
- 3.2 Random Oversampling . . . 4
- 3.3 Synthetic Minority Over-sampling Technique (SMOTE) . . . 4
- 3.4 Advanced SMOTE Variants . . . 5

4 The Data Leakage Trap & Leak-Proof Pipelines 5
- 4.1 The Catastrophe of Premature SMOTE . . . 5
- 4.2 The Solution: `imblearn` Pipelines . . . 6

5 Algorithmic Adjustments & The "Double Balancing" Pitfall 6

6 Comprehensive Implementation on `Heart.csv` 7
- 6.1 Step 1: Installation and Library Import . . . 7
- 6.2 Step 2: Inspecting and Visualizing Class Distribution . . . 7
- 6.3 Step 3: Building SMOTE-Integrated Pipelines . . . 7
- 6.4 Step 4: Updating Hyperparameter Grids . . . 8
- 6.5 Step 5: Executing Leak-Proof Cross-Validation . . . 9

7 Advanced Clinical Evaluation Metrics 9
- 7.1 Decomposing the Confusion Matrix . . . 9
- 7.2 Interpreting the Results: SMOTE vs. Baseline . . . 10

8 Model Selection & Policy Recommendations 11
- 8.1 The Clinical Trade-off . . . 11
- 8.2 Recommended Final Model . . . 11

9 Capstone Preparation: The Algorithmic Audit 12

10 Glossary of Key Mathematical Terms 12

11 References & Further Reading 13

1 The Clinical Reality of Class Imbalance

1.1 The Accuracy Paradox

In balanced datasets, accuracy is a reasonable proxy for model performance. However, in clinical datasets, the target variable is often highly skewed. Consider a dataset where only 5% of patients have a rare cardiovascular anomaly. A "dumb" model that simply predicts “No Disease” for every patient will achieve 95% accuracy.

If a hospital administrator evaluates this model based solely on accuracy, they might deploy it, leading to a catastrophic failure: 100% of the sick patients are missed. This is the **Accuracy Paradox**.

The Clinical Cost of Errors

In medical diagnostics, errors are not symmetric:

- **False Negative (Type II Error):** The model predicts “Healthy,” but the patient has the disease. *Cost:* Delayed treatment, acute myocardial infarction (MI), stroke, or mortality. **Catastrophic.**
- **False Positive (Type I Error):** The model predicts “Disease,” but the patient is healthy. *Cost:* Unnecessary, expensive, and invasive follow-up tests (e.g., angiography), patient anxiety, and wasted healthcare resources. **Moderate to High.**

Kenyan Clinical Context

In Kenyan county hospitals, non-communicable diseases (NCDs) are rising, but resources for advanced diagnostics (like echocardiograms or angiograms) are scarce. A model with high False Positives will overwhelm the limited cardiology referral system at facilities like Kenyatta National Hospital (KNH). Conversely, a model with high False Negatives will send high-risk patients home, leading to preventable deaths. Therefore, we must explicitly optimize for **Recall (Sensitivity)** while maintaining acceptable **Precision**.

1.2 Why Standard Algorithms Fail on Imbalanced Data

Most machine learning algorithms (including Decision Trees, Random Forests, and SVMs) are designed to minimize **overall error** or maximize **overall likelihood**.

- **Decision Boundaries:** Algorithms will naturally push the decision boundary toward the minority class to correctly classify the vast majority of the majority class, sacrificing the minority class to achieve a lower global loss.
- **Gradient Descent:** In neural networks or gradient boosting, the gradients from the majority class dominate the weight updates, drowning out the signal from the rare minority instances.

2 Mathematical Foundations of Imbalanced Learning

2.1 Prior Probability Shift

Let $P(y = 1)$ be the prior probability of the positive (minority) class. In an imbalanced dataset, $P(y = 1) \ll P(y = 0)$. Bayes' Theorem states that the posterior probability is:

$$P(y = 1|x) = \frac{P(x|y = 1)P(y = 1)}{P(x)}$$

When $P(y = 1)$ is extremely small, $P(y = 1|x)$ will also be small for most x, causing the classifier to default to predicting $y = 0$ unless the likelihood $P(x|y = 1)$ is overwhelmingly large.

2.2 The Goal of Balancing

The goal of data balancing is not to change the underlying data-generating process, but to alter the **training distribution** so that the algorithm learns a decision boundary that reflects the **true cost of errors** rather than the **prior class frequencies**. We aim to create a synthetic training environment where $P_{train}(y = 1) \approx P_{train}(y = 0)$, forcing the model to pay equal attention to both classes.

3 Data-Level Resampling Techniques

3.1 Random Undersampling

This technique randomly removes instances from the majority class until the classes are balanced.

- **Pros:** Reduces training time and memory footprint; prevents the model from being overwhelmed by the majority class.
- **Cons: Loss of valuable information.** It may discard borderline or critical majority examples that help define the true decision boundary, leading to a loss of specificity.

3.2 Random Oversampling

This technique randomly duplicates instances from the minority class until the classes are balanced.

- **Pros:** No loss of information; simple to implement.
- **Cons: High risk of overfitting.** The model will memorize the exact duplicated instances, creating overly specific, fragmented decision regions around the minority samples.

3.3 Synthetic Minority Over-sampling Technique (SMOTE)

Proposed by Chawla et al. (2002), SMOTE overcomes the overfitting problem of random oversampling by generating **synthetic** (new) examples of the minority class rather than simply duplicating existing ones.

SMOTE Mathematical Formulation

For each minority class instance x_i:

1. Find its k nearest neighbors within the *minority class* using Euclidean distance. Let

one of these neighbors be x_{zi}.

2. Generate a synthetic sample x_{new} along the line segment connecting x_i and x_{zi}:

$$x_{new} = x_i + \lambda \times (x_{zi} - x_i)$$

where λ is a random number drawn from a uniform distribution, $\lambda \sim U(0, 1)$.

Why this works: By interpolating between existing minority instances, SMOTE expands the decision region of the minority class without simply copying noise. It forces the classifier to draw broader, more generalized decision boundaries around the minority class.

3.4 Advanced SMOTE Variants

- **Borderline-SMOTE:** Only generates synthetic samples for minority instances that are near the decision boundary (i.e., their nearest neighbors include majority class instances). This focuses the model's attention on the "hard" examples. to clean up the decision boundary after SMOTE has potentially introduced overlapping synthetic samples.
- **SMOTENC:** Specifically designed for datasets with mixed categorical and numerical features (like our 'Heart.csv' dataset). Standard SMOTE calculates Euclidean distance, which is meaningless for categorical variables. SMOTENC uses a modified distance metric and mode-based interpolation for categorical features.

4 The Data Leakage Trap & Leak-Proof Pipelines

4.1 The Catastrophe of Premature SMOTE

The most common and devastating mistake in imbalanced learning is applying SMOTE to the **entire dataset** *before* splitting it into training and validation/test sets.

The Data Leakage Mechanism

If SMOTE is applied globally:

1. The algorithm calculates nearest neighbors across the *entire* dataset.
2. A synthetic sample x_{syn} might be generated using a training instance x_{train} and a test instance x_{test} as neighbors.
3. Consequently, information from the test set has literally been baked into the training set.
4. **Result:** The model is evaluated on data it has already "seen" in a synthetic form. Performance metrics (especially ROC-AUC and Recall) will be wildly, unrealistically inflated, leading to catastrophic failure in production.

4.2 The Solution: `imblearn` Pipelines

To prevent leakage, SMOTE must **only** be applied to the training folds during cross-validation. The standard `sklearn.pipeline.Pipeline` does not support resampling steps natively because it assumes all steps are transformers that modify X without changing the number of rows.

The `imbalanced-learn` (`imblearn`) library provides `imblearn.pipeline.Pipeline`, which is specifically designed to handle resampling safely.

Leak-Proof Execution Flow

During `RandomizedSearchCV` with an `ImbPipeline`:

1. The data is split into 5 folds.
2. For Fold 1: The pipeline calls `fit_transform` on the 4 training folds.
3. The `preprocessor` imputes and scales the 4 training folds.
4. **SMOTE is dynamically applied ONLY to these 4 preprocessed training folds**, generating synthetic samples to balance them.
5. The classifier is trained on this augmented training set.
6. The pipeline calls `transform` (NOT `fit_transform`) on the 1 validation fold. **SMOTE is skipped.** The validation fold remains completely original and untouched.
7. The model is evaluated on this pristine validation fold.

This process repeats for all 5 folds, guaranteeing zero data leakage.

5 Algorithmic Adjustments & The "Double Balancing" Pitfall

In previous sessions, we handled imbalance by passing `class_weight='balanced'` to tree-based models or `scale_pos_weight` to XGBoost. These parameters adjust the internal loss function to penalize minority class errors more heavily.

The "Double Balancing" Trap

If you use SMOTE (which balances the data at the *data level* by creating synthetic samples) **AND** you also tell the model to apply class weights (which balances the data at the *algorithmic level*), the model will **over-compensate**.

The Result: The model becomes overly biased toward the synthetic minority class. This leads to a massive spike in False Positives and a severe drop in Precision, as the model aggressively flags almost everything as positive to avoid the heavily penalized minority class error.

The Solution: When using SMOTE within an `ImbPipeline`, you **must remove** `class_weight` and `scale_pos_weight` parameters from the model definitions. SMOTE handles the balancing; the model should just learn from the balanced data with uniform weights.

6 Comprehensive Implementation on `Heart.csv`

6.1 Step 1: Installation and Library Import

We must install and import the specialized `imbalanced-learn` library.

```
# Install the library (run once in your environment)
# !pip install imbalanced-learn

# Import SMOTE and the specialized imblearn Pipeline
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline
```

Listing 1: Installing and Importing imbalanced-learn

6.2 Step 2: Inspecting and Visualizing Class Distribution

Before applying SMOTE, we must establish our baseline class distribution in the training set.

```
# Initialize SMOTE with a fixed random state for reproducibility
# k_neighbors=5 is the default, looking at 5 nearest minority neighbors
smote = SMOTE(random_state=42, k_neighbors=5)

# 1. Check original class distribution
print("       Original Training Set Class Distribution:")
print(y_train.value_counts())
print("-" * 50)

# 2. Manually apply SMOTE to the preprocessed training data
# just to visualize the result (Note: This is for visualization only,
# the actual pipeline handles this safely during CV)
X_train_prep = preprocessor.fit_transform(X_train)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train_prep, y_train)

print("       Balanced Training Set Class Distribution (after SMOTE):")
print(pd.Series(y_train_resampled).value_counts())
```

Listing 2: Inspecting Original Class Distribution

Code Explanation:

- `random_state=42`: Ensures the random neighbor selection and λ generation are reproducible.
- `fit_resample()`: This is the core SMOTE function. It takes the features and labels, and returns the augmented dataset where the minority class has been synthetically expanded to match the majority class count (e.g., from 111 to 131 instances).

6.3 Step 3: Building SMOTE-Integrated Pipelines

Here, we rebuild our pipelines using `ImbPipeline`. Notice the explicit removal of `class_weight`.

```
# Decision Tree Pipeline (NO class_weight)
dt_pipeline_smote = ImbPipeline(steps=[
    ('preprocessor', preprocessor),
    ('smote', SMOTE(random_state=42)),
    ('dt', DecisionTreeClassifier(random_state=42))
```

```
])

# Random Forest Pipeline (NO class_weight)
rf_pipeline_smote = ImbPipeline(steps=[
    ('preprocessor', preprocessor),
    'smote', SMOTE(random_state=42)),
    ('rf', RandomForestClassifier(random_state=42, n_jobs=-1))
])

# XGBoost Pipeline (scale_pos_weight neutralized to 1)
xgb_pipeline_smote = ImbPipeline(steps=[
    ('preprocessor', preprocessor),
    ('smote', SMOTE(random_state=42)),
    ('xgb', xgb.XGBClassifier(random_state=42, eval_metric='logloss',
                              n_jobs=-1, scale_pos_weight=1))
])
```

Listing 3: Defining SMOTE-augmented, Leak-Proof Pipelines

Code Explanation: The order is critical: `preprocessor` → `smote` → `classifier`. For XGBoost, we explicitly set `scale_pos_weight=1` (the default for balanced data) to override any previous dynamic weight calculations, ensuring SMOTE is the sole balancing mechanism.

6.4 Step 4: Updating Hyperparameter Grids

We must scrub the hyperparameter grids of any class-weighting parameters to prevent the grid search from accidentally selecting them.

```
# Updated Grid for Random Forest
param_dist_rf_smote = {
    'rf__n_estimators': [100, 200, 300, 400, 500],
    'rf__max_depth': [None, 5, 10, 15, 20, 25],
    'rf__min_samples_split': [2, 5, 10, 15],
    'rf__min_samples_leaf': [1, 2, 4, 6],
    'rf__max_features': ['sqrt', 'log2', None],
    'rf__criterion': ['gini', 'entropy', 'log_loss']
    # 'rf__class_weight' has been REMOVED
}

# Updated Grid for XGBoost
param_dist_xgb_smote = {
    'xgb__n_estimators': [100, 200, 300, 400, 500],
    'xgb__max_depth': [2, 3, 4, 5, 6],
    'xgb__learning_rate': [0.01, 0.03, 0.05, 0.1, 0.2],
    'xgb__subsample': [0.7, 0.8, 0.9, 1.0],
    'xgb__colsample_bytree': [0.7, 0.8, 0.9, 1.0],
    'xgb__min_child_weight': [1, 3, 5, 7],
    'xgb__gamma': [0, 0.1, 0.3, 0.5, 1],
    'xgb__reg_alpha': [0, 0.01, 0.1, 1],
    'xgb__reg_lambda': [0.1, 1, 5, 10]
    # 'xgb__scale_pos_weight' has been REMOVED
}
```

Listing 4: Updated Hyperparameter Grids (Removed class_weight)

6.5 Step 5: Executing Leak-Proof Cross-Validation

The execution loop remains identical to previous weeks, but the magic happens under the hood within the `ImbPipeline`.

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
print("\n    Starting RandomizedSearchCV with SMOTE (n_iter=30, 5-Fold Stratified)...")

results_ensemble_smote = {}
training_times_smote = {}

for name, pipeline, params in ensemble_pipelines_smote:
    start = time.time()

    search = RandomizedSearchCV(
        estimator=pipeline,
        param_distributions=params,
        n_iter=30,
        cv=cv,
        scoring='roc_auc',
        n_jobs=-1,
        random_state=42,
        refit=True
    )

    # SMOTE is applied safely ONLY to the training folds inside this call!
    search.fit(X_train, y_train)

    training_times_smote[name] = time.time() - start
    results_ensemble_smote[name] = search

    print(f"    {name} trained. Best ROC-AUC: {search.best_score_:.4f} | Time: {training_times_smote[name]:.2f}s")
```

Listing 5: Executing RandomizedSearchCV Safely

7 Advanced Clinical Evaluation Metrics

After training, we must evaluate the models on the **completely untouched, imbalanced hold-out test set**.

The Golden Rule of Evaluation

Never apply SMOTE, scaling, or imputation to the hold-out test set. The test set must reflect the real-world, imbalanced distribution of patients arriving at the hospital.

7.1 Decomposing the Confusion Matrix

```
eval_metrics_smote = {}
for name, search in results_ensemble_smote.items():
    model = search.best_estimator_

    # Predict on the PURE, unbalanced, unseen test set
```

```
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1]

    cm = confusion_matrix(y_test, y_pred)
    tn, fp, fn, tp = cm.ravel()

    # Calculate clinical metrics
    recall = recall_score(y_test, y_pred, zero_division=0)
    precision = precision_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)
    roc_auc = roc_auc_score(y_test, y_prob)

    # False Negative Rate (Crucial for clinical context)
    fnr = fn / (fn + tp) if (fn + tp) > 0 else 0

    metrics = {
        'Accuracy': accuracy_score(y_test, y_pred),
        'Precision': precision,
        'Recall': recall,
        'F1-Score': f1,
        'ROC-AUC': roc_auc,
        'FNR': fnr
    }
    eval_metrics_smote[name] = {**metrics, 'y_pred': y_pred, 'y_prob': y_prob, '
    model': model, 'confusion_matrix': cm}
```

Listing 6: Hold-Out Test Set Evaluation Setup

7.2 Interpreting the Results: SMOTE vs. Baseline

When comparing the SMOTE-augmented models to our Week 5 baselines, we observe a distinct pattern:

- **Recall (Sensitivity) Surge:** SMOTE significantly boosts Recall across the board. For example, Extra Trees might jump to a massive **96.43%** Recall (FNR of only 3.57%). It misses very few sick patients because the decision boundary has been explicitly expanded around the minority class.
- **Precision Trade-off:** This increased sensitivity comes at a cost. Extra Trees' Precision might drop to 81.82%. Because the model is now highly sensitive to the minority class, it flags more healthy patients as sick (False Positives increase).
- **Random Forest Stability:** Random Forest typically maintains an excellent balance. Its Recall remains very high (e.g., 92.86%), but its Precision and ROC-AUC remain superior to Extra Trees, making it more reliable overall.

Why PR-AUC Matters More Than ROC-AUC Here

In imbalanced clinical datasets, the number of True Negatives (TN) is very large.

$$\text{Specificity} = \frac{TN}{TN + FP}$$

Because TN is so large, FP can increase significantly without drastically lowering Specificity.

Consequently, the False Positive Rate ($1-\text{Specificity}$) on the X-axis of the ROC curve remains artificially low, making the ROC-AUC look deceptively good.
The **Precision-Recall (PR) Curve**, however, ignores True Negatives entirely:

$$\text{Precision} = \frac{TP}{TP + FP}$$

As the model generates more False Positives (a direct consequence of SMOTE), Precision drops sharply. Therefore, **PR-AUC is a much more honest and informative metric** for evaluating SMOTE-augmented clinical models than ROC-AUC.

8 Model Selection & Policy Recommendations

8.1 The Clinical Trade-off

Choosing the final model is not a purely mathematical exercise; it is a **policy decision** based on the hospital's risk tolerance.

Threshold Selection for Policy

- **Scenario A: "Miss No Cases at All Costs" (e.g., Primary Care Screening by CHPs)**
  Choose: Extra Trees.
  Justification: Lowest FNR (3.57%). We accept a higher False Positive rate (more healthy patients referred) to ensure that virtually no sick patient is sent home. The cost of an unnecessary referral is far lower than the cost of a missed myocardial infarction.

- **Scenario B: "Optimal Resource Allocation with High Reliability" (e.g., Specialist Cardiology Referral)**
  Choose: Random Forest.
  Justification: Best F1-Score and ROC-AUC/PR-AUC. It provides an exceptional balance. A Recall of 92.86% is still clinically excellent (identifying 26 out of 28 actual disease cases), but it avoids the severe precision drop seen in Extra Trees, ensuring the model remains trustworthy and does not overwhelm the limited angiography resources at the county hospital.

8.2 Recommended Final Model

For this study, the **SMOTE-augmented Random Forest** is selected as the overall final model.

- **Justification:** It achieved the highest hold-out ROC-AUC and PR-AUC, indicating superior discriminative power without overfitting to synthetic noise.
- Its Recall of 92.86% is exceptionally high for a clinical setting.
- It avoids the severe precision drop seen in Extra Trees, ensuring the model remains cost-effective and trustworthy for county health facilities.

9 Capstone Preparation: The Algorithmic Audit

Exercise: Defending Your Model

Imagine you are deploying the Extra Trees model to a county hospital. The hospital administrator asks: *"Why did the model flag this 45-year-old female patient with normal cholesterol as high risk, and why should we trust it over a simple rule?"*

1. **Explain the trade-off:** Articulate that Extra Trees was chosen specifically to minimize False Negatives (Recall = 96.43%), accepting a slightly higher False Positive rate to ensure no sick patient is missed.
2. **Propose an XAI solution:** Explain that you will use SHAP (SHapley Additive exPlanations) values to show exactly which features (e.g., atypical `ChestPain` type, elevated `Oldpeak`) drove this specific prediction, overriding the "normal cholesterol" assumption.
3. **Draft a response:** Write a 2-sentence response to the administrator that balances technical honesty with clinical reassurance, emphasizing that the model acts as a "safety net" rather than a definitive diagnosis.

10 Glossary of Key Mathematical Terms

SMOTE Synthetic Minority Over-sampling Technique. Generates synthetic minority instances by interpolating between a seed instance and its k nearest minority neighbors.

Data Leakage The inadvertent inclusion of information from the validation or test set into the training process, leading to optimistically biased performance estimates.

ImbPipeline A specialized pipeline from the `imbalanced-learn` library that safely applies resampling techniques only to the training folds during cross-validation.

Double Balancing The detrimental practice of applying both data-level balancing (SMOTE) and algorithmic-level balancing (`class_weight`), leading to severe over-compensation and plummeting Precision.

PR-AUC Area Under the Precision-Recall Curve. A superior metric to ROC-AUC for imbalanced datasets, as it focuses exclusively on the performance regarding the positive (minority) class, ignoring the inflated True Negative count.

False Negative Rate (FNR) The proportion of actual positive cases that were incorrectly classified as negative. $FNR = 1 - \text{Recall}$. The most critical metric in clinical safety.

11 References & Further Reading

- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). *SMOTE: synthetic minority over-sampling technique.* Journal of artificial intelligence research, 16, 321-357.
- He, H., & Garcia, E. A. (2009). *Learning from imbalanced data.* IEEE Transactions on knowledge and data engineering, 21(9), 1263-1284.
- Saito, T., & Rehmsmeier, M. (2015). *The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets.* PloS one, 10(3), e0118432.
- `imbalanced-learn` Documentation: `https://imbalanced-learn.org/stable/`
- Mediacrest Training College. (2026). *Advanced Data Analytics & Machine Learning Curriculum.*

Mediacrest Training College – Nairobi, Kenya

Advanced Data Analytics & Machine Learning · Week 6, Day 1

Trainer: Victor Wandera Lumumba, MSc. · September 2026